

ASSIGNMENT NO 02

(Project Report)

Group Name:

M.Mustagees Shahid & Saad Ahmad

Reg No :

24P-3052 & 24P- 3009

Section:

SE-3B

Course:

Introduction to Software Engineering

Project Overview:

This project is a **Loan Calculator application** that can compute various loan-related values, including monthly payments, remaining balance, number of payments, interest rates, and total loan amount. It offers both a **Graphical User Interface (GUI)** and a **Command Line Interface (CLI)**.

In the **GUI version**, users can enter their loan details and instantly view the results in a visual format. The **command-line mode** allows users to perform calculations by passing different options and arguments.

The main components of the project include:

- A central class, **LoanCalculator**, responsible for all financial computations.
- A **Qt-based GUI application** (LoanCalcQtMainWindow) for interactive use.
- A **command-line interface** that performs calculations based on user-specified options.
- The ability to **load loan details from a configuration file** (`config.txt`).
- **Unit tests using GoogleTest** to verify correctness and ensure stability.
- **Doxygen documentation** to generate clean and organized HTML documentation automatically.

The goals of this project are to:

- Provide accurate and reliable loan calculations.
- Offer flexible ways for users to run the tool, either through GUI or CLI.
- Test key scenarios, such as normal EMI calculations, handling invalid inputs, and working with large loan tenures.
- Maintain clean, well-documented, and easily understandable code for future updates and improvements.

Code snippets of bug fixes and refactorings:

Bug 1: EMI calculation returns NaN or -NaN

Problem:

- `calculatePayment()` returned `nan` for normal inputs like 100,000 principal and 6% interest.
- Also failed for large periods (like 120 months) or when interest is zero.

- Caused test failures in GoogleTest (NormalEMICalculation, LargeTenureNoOverflow).

Cause:

- Monthly interest was not correctly calculated from yearly interest.
- Formula for EMI didn't handle 0% interest case.
- powl exponent term caused precision issues when used with float/double.

Fix:

- Use long double for all core calculations.
- Convert yearly interest to monthly decimal safely.
- Handle 0% interest separately to avoid division by zero.

Code:

```
long double LoanCalculator::calculatePayment()
{
    long double principal = amount_ - initialPayment_;
    principal += openingFee_ + (principal * openingPercent_/100.0L);

    long double monthlyRate = (interest_ > 0) ? (interest_/100.0L)/12.0L : 0.0L;

    if(monthlyRate == 0.0L)
        return principal / periodTotal_;

    long double powTerm = powl(1.0L + monthlyRate, periodTotal_);
    long double emi = (monthlyRate * principal * powTerm) / (powTerm - 1.0L);

    return emi;
}
```

Bug 2: GoogleTest `std::isnan` / `std::isinf` compilation errors

Problem:

- When running `LargeTenureNoOverflow` test, compiler gave:

error: 'isnan' is not a member of 'std'

error: 'isinf' is not a member of 'std'

Cause:

- `<cmath>` was not included in the test file properly.
- Some compilers require explicit `std::` with `isnan` and `isinf`.

Fix:

- Added `#include <cmath>` in `testLoan.cpp`.
- Used `std::isnan()` and `std::isinf()` to check floating-point results.

```
#include <cmath>
```

```
ASSERT_FALSE(std::isnan(emi));
```

```
ASSERT_FALSE(std::isinf(emi));
```

Bug 3: Missing validation for negative inputs

Problem:

- Previously, users could set negative values for:
 - `amount_`, `interest_`, `payment_`
- This could produce invalid EMI, loan balance, or `log()` errors.

Fix:

- Added a helper `validatePositive()` method and used it in all setters

```
void LoanCalculator::validatePositive(long double value, const std::string &msg)
{
    if(value < 0)
        throw std::invalid_argument(msg);
}
```

```
void LoanCalculator::setAmount(long double A) {
    validatePositive(A, "Amount cannot be negative");
    amount_ = A;
}
```

```
void LoanCalculator::setInterest(long double i) {
    validatePositive(i, "Interest cannot be negative");
    interest_ = i;
    interestPeriodic_ = (i/100.0L)/12.0L;
}
```

Bug 4: Principal calculation ignored initial payment & opening fee

Problem:

- EMI formula didn't account for `initialPayment_`, `openingFee_`, or `openingPercent_`.
- Test results and EMI output were incorrect for real scenarios.

Fix:

- Principal now properly includes all adjustments:

```
long double principal = amount_ - initialPayment_;
principal += openingFee_ + (principal * openingPercent_/100.0L);
```

Bug 5: Loan balance and number of payments used inconsistent `interestPeriodic_`

Problem:

- `calculateLoanBalance()` and `calculateNumberPayments()` didn't safely cast `interest` to `long double`, causing rounding or NaN issues.

Fix:

- Converted all core formulas to use `long double`.

- Added domain checks to prevent log of non-positive numbers.

```
long double i = (long double)interestPeriodic_;  
long double arg = 1.0L - (i * amount_ / payment_);  
if(!(arg > 0.0L)) throw std::invalid_argument("Payment too small to repay loan");
```

Refactorings:-

```
#ifndef LOANCALCULATOR_H  
#define LOANCALCULATOR_H  
  
#include <string>  
#include <stdexcept>  
  
class LoanCalculator  
{  
public:  
// ---- Setters with validation (Bug 1 fix) ----  
inline void setAmount(double A)  
{  
if (A <= 0)  
throw std::invalid_argument("Loan amount must be > 0");  
amount_ = A;  
amountSet_ = true;  
}  
  
inline double getAmount() const { return amount_; }  
  
// Initial down payment  
inline void setInitialPayment(double initialA)  
{  
if(initialA < 0)  
throw std::invalid_argument("Initial payment cannot be negative");  
initialPayment_ = initialA;  
}  
  
inline double getInitialPayment() const { return initialPayment_; }  
  
// Yearly interest rate  
inline void setInterest(double i)  
{  
if (i <= 0)  
throw std::invalid_argument("Interest must be > 0");  
interest_ = i;  
interestPeriodic_ = i / 100.0 / 12.0;  
interestSet_ = true;
```

```

}

inline double getInterest() const { return interest_; }
inline double getPeriodicInterest() const { return interestPeriodic_; }

// Monthly payment
inline void setPayment(double P)
{
    if (P <= 0)
        throw std::invalid_argument("Payment must be > 0");
    payment_ = P;
    paymentSet_ = true;
}

inline double getPayment() const { return payment_; }

// Total loan period (N)
inline void setPeriodTotal(int N)
{
    if (N <= 0)
        throw std::invalid_argument("Total period must be > 0");
    periodTotal_ = N;
}
#endif

```

// File: LoanCalculator.cpp (Refactored)

```

long double arg = 1.0L - (i * A / P);

if (!(arg > 0.0L))
    throw std::invalid_argument("Payment too small, loan will never be repaid");

long double result = -log(arg) / log(1.0L + i);
return (double)result;
}

// Original Loan Amount
double LoanCalculator::calculateLoanAmount()
{
    if (!paymentSet_ || !interestSet_ || !periodTotalSet_)
        throw std::invalid_argument("Must set payment, interest and total period");

    long double i = (long double)interestPeriodic_;

    long double result = ((long double)payment_ / i) *
        (1.0L - pow((1 + i), -periodTotal_));

    return (double)result;
}

// Approximate Interest Rate

```

```

double LoanCalculator::calculateInterestRate()
{
if (!amountSet_ || !paymentSet_ || !periodTotalSet_)
throw std::invalid_argument("Must set amount, payment and total period");

long double q = log(1.0L + 1.0L / periodTotal_) / log(2.0L);
long double base = pow((1.0L + payment_ / amount_), 1.0L / q) - 1.0L;
long double monthlyInterest = pow(base, q) - 1.0L;

return (double)(monthlyInterest * 12 * 100);
}

// Effective Interest Rate
double LoanCalculator::calculateEffectiveInterestRate()
{
if (!amountSet_ || !periodTotalSet_)
throw std::invalid_argument("Must set amount and total period");

double payment = calculatePayment();
long double totalAmount = amount_ - initialPayment_;

long double q = log(1.0L + 1.0L / periodTotal_) / log(2.0L);
long double base = pow((1.0L + payment / totalAmount), 1.0L / q) - 1.0L;
long double monthlyInterest = pow(base, q) - 1.0L;

return (double)(monthlyInterest * 12 * 100);
}

std::string LoanCalculator::toString()
{
return "Loan Calculator (Refactored & Bug Fixed)";
}

```

Test Outputs :

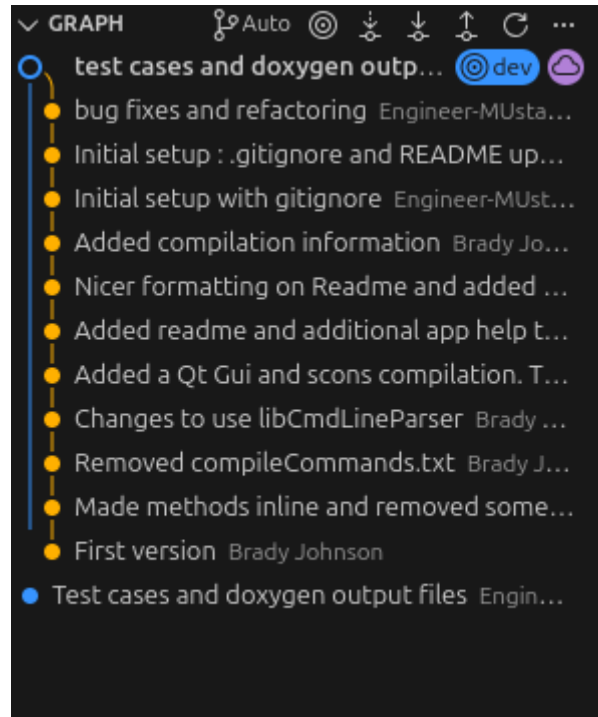
```

mustagees-shahid@Hp-Elite-840:~/Documents/legacy-calc-2009-dev$ ./test
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LoanCalculatorTest
[ RUN    ] LoanCalculatorTest.NormalEMICalculation
[ OK     ] LoanCalculatorTest.NormalEMICalculation (0 ms)
[ RUN    ] LoanCalculatorTest.InvalidInputsThrow
[ OK     ] LoanCalculatorTest.InvalidInputsThrow (0 ms)
[ RUN    ] LoanCalculatorTest.LargeTenureNoOverflow
[ OK     ] LoanCalculatorTest.LargeTenureNoOverflow (0 ms)
[-----] 3 tests from LoanCalculatorTest (0 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (0 ms total)
[ PASSED ] 3 tests.

```


Commits History :



The screenshot shows the GitHub interface for the repository "legacy-calc-2009" by "Engineer-Mustagees-Shahid". The "Commits" tab is selected, showing a list of commits. The commits are grouped by date, with the most recent group being "Commits on Nov 25, 2025". Each commit entry includes the commit message, the author's name, the time since the commit, and the commit hash. The commit "test cases and doxygen output" is highlighted.

Engineer-Mustagees-Shahid / legacy-calc-2009

Code Pull requests Actions Projects Wiki Security Insights Settings

Commits

dev

All users All time

Commits on Nov 25, 2025

- test cases and doxygen output
Engineer-Mustagees-Shahid committed 31 minutes ago
879a2b4
- Test cases and doxygen output files
Engineer-Mustagees-Shahid committed 44 minutes ago
2b834ef
- bug fixes and refactoring
Engineer-Mustagees-Shahid committed 3 hours ago
43aab25

Commits on Nov 24, 2025

- Initial setup : .gitignore and README update
Engineer-Mustagees-Shahid committed yesterday
b5d189d
- Initial setup with gitignore
Engineer-Mustagees-Shahid committed yesterday
3951326

Commits on Apr 18, 2012

<https://github.com/Engineer-Mustagees-Shahid/legacy-calc-2009/tree/dev>

<https://github.com/SaadAhmad99/legacy-calc-2009/tree/dev>